

◆ 1. Execution Context & Hoisting

◆ Concepts

- Execution Context (Global, Function, Eval)
- Creation & Execution Phase
- Hoisting (Variables & Functions)
- Temporal Dead Zone (TDZ)

◆ Keywords

- `var`, `let`, `const`:
 - Scope
 - Re-declaration
 - Hoisting
 - Initialization

◆ Common Errors

- ReferenceError (e.g. TDZ)
- SyntaxError
- TypeError

◆ Practical Implementations

- Scope chains
 - Hoisting visual questions
-

◆ 2. Closures

◆ Concepts

- Closure definition
- Lexical Environment
- Scopes: Global, Local, Block

◆ Implementation Examples

- Counter function
 - Private variables
 - Function factories
-

◆ 3. Callbacks & Asynchronous JavaScript

◆ Callback

- Callback function definition
- **Callback Hell:**
 - Pyramid of doom
 - Real-world example (e.g., nested API calls)

◆ Solving Callback Hell

- **Promises**
 - **Async/Await**
-

◆ 4. Promises

◆ Core Concepts

- Promise syntax: `resolve`, `reject`, `then`, `catch`, `finally`
- Error handling in promises

◆ Important Methods

- `Promise.all()`
- `Promise.race()`
- `Promise.allSettled()`
- `Promise.any()`

◆ Real-World Use Cases

- Concurrent API calls (e.g., fetching weather and news)
-

◆ 5. Async JavaScript

◆ Event Loop Model

- Call Stack
- Web APIs
- Callback Queue
- Microtask Queue (Promises, MutationObservers)

◆ `async` & `await`

- How they work under the hood
 - Limitations (e.g., parallel execution)
-

◆ 6. Axios (or Fetch) API

◆ HTTP Methods

- `GET`, `POST`, `PUT`, `PATCH`, `DELETE`

◆ Use Cases

- CRUD operations
 - Axios interceptors
 - Error handling
-

◆ 7. Call, Apply, Bind

◆ Concepts & Use Cases

- `call()`: Immediate execution with context
- `apply()`: Like call, with arguments as array
- `bind()`: Returns a new function

◆ Polyfill Implementations (Read about it)

- Create your own `call`, `apply`, and `bind`
-

◆ 8. Event Delegation, Debouncing, Throttling

◆ Event Delegation

- Event bubbling
- Delegating handlers for performance

◆ Debouncing

- Delays function execution
- Use case: Search input

◆ Throttling

- Limits function call frequency
 - Use case: Window resize, scroll events
-

◆ 9. `this` Binding

◆ Context Binding

- Global, Object, Class, Arrow Functions Arrow functions & lexical `this`
 - Explicit binding: `call`, `apply`, `bind`
-

◆ 10. Array, Object, String Methods

◆ Arrays

- `.map()`, `.filter()`, `.reduce()`, `.find()`, `.forEach()`, `.some()`, `.every()`

◆ Objects

- `Object.keys()`, `Object.values()`, `Object.entries()`, destructuring, spread/rest

◆ Strings

- `replace()`, `replaceAll()`, `toUpperCase()`, `toLowerCase()`, `split()`, `slice()`, `splice()`, `includes()`
-

◆ 11. OOP in JavaScript

◆ Core Pillars

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

◆ JS-Specific Topics

- Class syntax
- Constructor functions
- `super` keyword
- `static` methods
- Prototype & Prototype Chain

◆ Custom Implementations

- Custom `.reduce()` method
 - Inheritance via constructor or class
-

◆ 12. Functions in JavaScript

◆ Types

- First-class functions
 - Higher-order functions
 - Callback functions
 - Arrow functions (no `this`, `arguments`)
-

◆ 13. Spread & Rest Operators

◆ Spread (...)

- Cloning arrays/objects
- Merging

◆ Rest

- Handling variable number of arguments
 - Use in destructuring
-

◆ 14. Currying

◆ Concepts

- Breaking function into series of unary functions

◆ Implementations

- Basic currying
 - **Infinite currying**
-

◆ 15. Browser APIs

◆ Storage

- `localStorage` vs `sessionStorage` vs `cookies`

◆ Status Codes

- `200 OK`, `201 Created`, `204 No Content`
- `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`
- `500 Internal Server Error`, `503 Service Unavailable`